Davante

17

PROGRAMACIÓN

Interfaces gráficas de usuario

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Crear un proyecto con interfaz gráfica 5

/ 3. Elementos gráficos básicos 6

3.1. Botones: JButton 6

3.2. Elección múltiple: JCheckBox 7

3.3. Botón de radio: JRadioButton y ButtonGroup 7

3.4. Lista de valores: JComboBox 7

/ 4. Tablas 8

/ 5. Layouts 9

5.1. Flowlayout y GridLayout 10

/ 6. Ventanas de información 10

/ 7. Caso práctico 1: "Problemas de adaptación a la ventana" 11

/ 8. Ventanas de peligro y error 12

/ 9. Ventana personalizada y preguntas 14

/ 10. Ventanas para apertura y cierre de ficheros 15

/ 11. Caso práctico 2: "Abuso de los menús" 16

/ 12. Menús 17

ÍNDICE

/ 13. Creación y comunicación entre ventanas 18

/ 14. Resumen y resolución del caso práctico de la unidad 19

/ 15. Bibliografía 20

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Elaborar programas con interfaz gráfica.

Conocer los diferentes elementos de la interfaz gráfica.

Gestionar interfaces gráficas.

Realizar programas con más de una ventana gráfica.

Conocer los diferentes elementos de la interfaz gráfica.

Trabajar con eventos.

/ 1. Introducción y contextualización práctica

En esta unidad vamos a ver cómo podemos crear un proyecto en NetBeans con interfaz gráfica. Para ello, vamos a identificar los elementos más básicos con los que podremos hacer la gran mayoría de las interfaces que necesitemos. Además, veremos cómo podemos utilizarlos, hacer que se adapten a las interfaces y darles una funcionalidad concreta, por ejemplo, que al pulsar un botón haga algo.



Fig.1. Interfaz gráfica.

A continuación se tratará el uso de ventanas de diálogo que ya están implementadas y para las cuales no será necesario realizar su diseño de cero, por ejemplo, para la selección de ficheros. Además de las ventanas ya implementadas, se verá cómo crear varias ventanas en una misma aplicación y hacer que conecten entre ellas añadiendo menús.

A continuación, se presenta un caso práctico a través del cual podremos aproximarnos de forma aplicada a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio Intro. "Uso de la interfaz gráfica avanzado"

http://bit.ly/3ogic8z

/ 2. Crear un proyecto con interfaz gráfica

Hasta el momento, nuestros programas siempre han funcionado de la misma forma, pidiendo información al usuario para que la introduzca por teclado, procesando dicha información y mostrando por pantalla el resultado del procesamiento.

Ahora vamos a ver cómo podemos hacer que nuestros programas tengan una interfaz gráfica, donde podremos colocar elementos para que el usuario interactúe con ellos, tales como botones, cajas de texto, etiquetas, tablas, etc. A partir de este momento, nuestros programas dejarán de usar la consola para pasar a tener una interfaz gráfica totalmente funcional.

Para la creación de las interfaces gráficas en Java, vamos a utilizar la **biblioteca SWING**, la cual nos va a proveer de todo lo necesario para poder crear ventanas totalmente funcionales de una forma rápida y sencilla. Esta biblioteca ya viene integrada en la propia **JDK de Java**, así que no tendremos que instalar nada para poder utilizarla.

Para crear un **proyecto con interfaz gráfica** seguiremos los siguientes pasos:

1. Pulsamos en nuevo proyecto y seleccionamos 'Java application'.

2. Ponemos el nombre de nuestro proyecto, elegimos dónde queremos que se guarde y, donde pone 'Createmain class', lo tenemos que deseleccionar. De esta forma no creará la clase principal y tendremos un proyecto vacío.

3. Una vez hecho esto, en el paquete principal (que estará vacío), pulsamos con el botón derecho y elegimos Nuevo y 'JFrame form'. Esto es una ventana de interfaz gráfica.

4. Le ponemos el nombre a nuestra ventana. Lo que vamos a crear es una clase que hereda de javax.swing. Jframe, por lo que el nombre de las ventanas seguirá la nomenclatura de las clases.

5. Ahora pulsamos en ejecutar el proyecto y nos aparecerá un diálogo diciendo que no hay clase principal, con la ventana que acabamos de crear (figura), pulsamos en aceptar y ya tendremos configurado el proyecto para que esta sea la clase que se ejecute en primer lugar, es decir, que tenga interfaz gráfica por defecto.

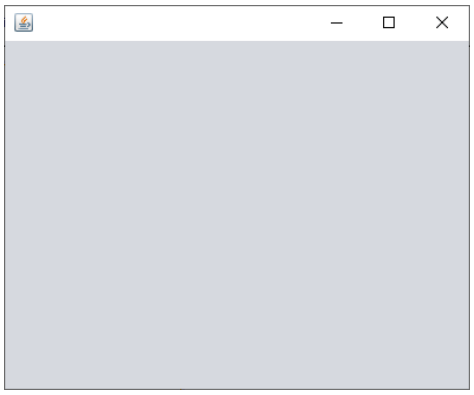


Fig.2. Proyecto vacío con interfaz gráfica.

¡! Sabías que...

Cuando queramos escribir código que se ejecute en nuestra aplicación con interfaz gráfica, deberemos hacerlo en el constructor, justo debajo de la función initComponents(),

/ 3. Elementos gráficos básicos

Vamos a empezar a ver los elementos gráficos más básicos que podemos utilizar en nuestros proyectos:

- **JLabel**: Etiqueta empleada para mostrar un texto cualquiera en nuestras ventanas.
- **JTextField**: Caja de texto, donde vamos a poder escribir/introducir un texto cualquiera en nuestras ventanas.
- **JTextArea**: Otro tipo de caja de texto, donde vamos a poder escribir/introducir un texto cualquiera en nuestras ventanas, pero, a diferencia del JTextField, este puede tener varias líneas escritas.

Todos los elementos los vamos a poder encontrar en la paleta de componentes, y para poder ponerlos en nuestra ventana bastará con seleccionarlos y arrastrarlos. Inmediatamente después de arrastrarlos, se colocarán donde los hayamos soltado.

Con estos elementos podremos hacer dos operaciones: cambiarles el texto y obtener el texto que tienen.

Las funciones para cambiar y obtener el texto son:

- **getText()**: Esta función devuelve el texto que hay en el JLabel, JTextField o JTextArea.
- **setText(String texto)**: Esta función modifica el texto del JLabel, JTextField o JTextArea, modificándolo por el que le pasamos como parámetro.

3.1. Botones: JButton

Los JButton son **botones que podremos poner en nuestras ventanas gráficas para poder interactuar con ellos**. La idea es bastante simple, tenemos un botón que al pulsarlo ejecutará una acción.

Los botones, al igual que todos los elementos, se encuentran en la paleta de componentes, y para poder ponerlos en nuestra ventana basta con seleccionarlos y arrastrarlos donde queramos.

Para cambiar y obtener el texto del botón, tenemos los métodos getTexty setText, al igual que con los JLabel, JTextField y JTextArea.

Para hacer que un botón ejecute una acción al pulsarlo, tendremos que sobrecargar el evento actionPerformed, como se muestra en la imagen, lo cual conseguiremos haciendo doble clic sobre el botón deseado, abriéndose la ventana de código con una nueva función creada, la cual se ejecutará cuando se pulse el botón.

```
private void jButton1ActionPerformed(java.awt.event.ActionEvent evt) {
    // Aquí se escribirá el código que se ejecutará
    // al pulsar el botón
}
```

Fig.3. Action del botón.

Cada botón tendrá un evento actionPerformed diferente, siendo estos independientes unos de otros.

Vídeo 1. "Dando funcionalidad a un JButton"

https://bit.ly/3pJzrPQ

3.2. Elección múltiple: JCheckBox

Los JCheckBox son **cuadros de elección múltiple con los que podremos elegir varias opciones de una posible pregunta**. Estos elementos se usan principalmente en registros de usuarios.

Con los *checkbox* podremos cambiar y obtener el texto como hemos hecho con los elementos previos mediante el uso de los métodos getText y setText, al igual que con los JLabel, JTextField y JTextArea, y, además:

- Seleccionarlo/deseleccionarlo.
- Saber si está seleccionado o no.

Para el uso de estas operaciones contamos con los siguientes métodos:

- **isSelected()**: Esta función devuelve verdadero o falso según el checkbox esté seleccionado o no.
- **setSelected(boolean valor)**: Esta función cambia el estado del checkbox ha seleccionado o no seleccionado.

3.3. Botón de radio: JRadioButton y ButtonGroup

Los **JRadioButton** son los típicos **círculos de única elección** que nos podemos encontrar en cualquier programa o web de registro. Con los *radio button* podremos realizar las mismas operaciones que con los checkbox y, por tanto, los métodos serán los mismos.

Para hacer que solo se pueda seleccionar un único radio button, tendremos que utilizar los ButtonGroup. Una vez tengamos un grupo creado, podremos introducir en él los radio button con la propiedad buttongroup, haciendo que de ese grupo solo pueda haber uno activo.

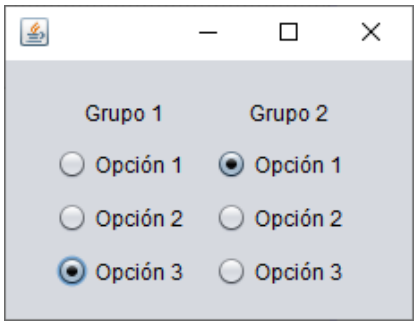


Fig.4. ButtonGroup.

3.4. Lista de valores: JComboBox

Los JComboBox son **listas desplegables en la que se puede elegir un único valor**. Las operaciones que se pueden realizar son:

- Agregar un elemento a la lista.
- Obtener el elemento que está seleccionado en la lista.

Para introducir valores en la lista del JComboBox, iremos a la pantalla de propiedades y seleccionamos la propiedad Model, que hace referencia a los datos que se incluyen en la lista. Estos valores, se introducirán separados por Intro.

Para agregar un elemento, utilizaremos el método addItem, al que le pasaremos un *string* con la línea que queremos agregar. Para saber qué elemento se ha seleccionado, se deberá usar el evento actionPerfomed (se ejecutará al seleccionar) junto al método getSelectedItem().toString() para obtener el elemento seleccionado.

> **¡! Sabías que...**
>
> Como primera letra del nombre del elemento pondremos un identificativo de su tipo, por ejemplo, para los JButton, una b, e inmediatamente después, la acción que realiza dicho elemento.

/ 4. Tablas

Los **JTable** son un componente más de la interfaz gráfica de usuario (GUI) que **permite mostrar datos en forma de tabla de manera ordenada**. Las acciones que podremos realizar son:

- Insertar filas.
- Eliminar filas.

Para la manipulación de elementos, tendremos que utilizar su modelo de la siguiente forma:

```
DefaultTableModel modelo = (DefaultTableModel) jtable.getModel();
```

Una vez lo tengamos, podremos:

- **Insertar una fila**: Usando el método addRow del modelo, al que le tendremos que pasar un *array* de string con los elementos que queremos que muestre. Estos deberán ir en el mismo orden de las columnas a mostrar.
- **Agregar una columna**: Para esto utilizaremos el método addColum del modelo, al que le pasaremos un string con el nombre de la nueva columna.
- **Eliminar una fila**: Usando el método removeRow del modelo, al cual le pasaremos el índice de la fila que queremos eliminar.
- **Saber qué fila hay seleccionada**: Emplearemos el método getSelectedRow o getSelectedRows si hay más de una, los cuales están en la tabla. El primero devolverá un entero con la fila seleccionada y el segundo, un array de enteros con las filas seleccionadas.
- **Saber qué columna hay seleccionada**: Para esto utilizaremos el método getSelectedColumn o getSelectedColumns si hay más de una, los cuales están en la tabla. El primero devolverá un entero con la columna seleccionada y el segundo, un array de enteros con las columnas seleccionadas.

A continuación se muestra una imagen (figura 5) con el código empleado para la creación de tablas.

```java
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;

public class DefaultTableModelExample extends JFrame {
    private DefaultTableModel model;
    private JTable table;

    public DefaultTableModelExample() {
        setTitle("Ejemplo de DefaultTableModel");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        // Crear un DefaultTableModel con datos y nombres de columnas iniciales
        model = new DefaultTableModel(
                new Object[][]{
                        {"Javi", 25, "M"},
                        {"Lucia", 30, "F"},
                        {"Fran", 35, "M"}
                },
                new String[]{"Nombre", "Edad", "Género"}
        );

        // Crear el JTable utilizando el DefaultTableModel
        table = new JTable(model);

        // Añadir el JTable a un JScrollPane para permitir desplazamiento si es necesario
        JScrollPane scrollPane = new JScrollPane(table);

        // Añadir botones para agregar y eliminar filas
        JButton addButton = new JButton("Agregar fila");
        addButton.addActionListener(e -> addRow());

        JButton deleteButton = new JButton("Eliminar fila");
        deleteButton.addActionListener(e -> deleteRow());

        JPanel buttonPanel = new JPanel();
        buttonPanel.add(addButton);
        buttonPanel.add(deleteButton);

        // Añadir componentes al JFrame
        getContentPane().add(scrollPane, BorderLayout.CENTER);
        getContentPane().add(buttonPanel, BorderLayout.SOUTH);

        pack();
        setLocationRelativeTo(null); // Centrar la ventana en la pantalla
    }
```

Fig.5. Creación de tablas.

Vídeo 2. "Introducir datos en un JTable"
https://bit.ly/38cYvJg

/ 5. Layouts

Ya hemos visto que tenemos muchos elementos para poder agregar a las ventanas. Cuando los vamos agregando, estos se colocan en el mismo sitio al cual los hemos arrastrado, pero ¿qué ocurre cuando cambiamos el tamaño de la ventana?

Cuando hagamos esto, los elementos se van a quedar en el mismo sitio donde estaban, no adaptándose al nuevo tamaño de la ventana. Para solucionarlo, tenemos los *layouts*, que van a hacer que esta tarea se automatice.

Existen **diferentes tipos de layouts**, y cada uno va a **colocar los elementos de una forma diferente**, ayudándonos a crear ventanas con los elementos organizados de diferentes formas.

Antes de nada, tendremos que ver un nuevo elemento, los **JPanel**. Estos permitirán **poner un panel dentro de la pantalla para agregarle elementos dentro**, exactamente de la misma forma que hemos estado haciendo hasta ahora. Podremos colocar tantos como queramos en nuestras ventanas.

Para cambiar un layout, en el panel del árbol de elementos, pulsaremos, con el botón derecho, sobre un panel y elegiremos la opción de cambiar layout, aparecerán todos los disponibles.

Audio 1. "Recomendación sobre layouts"
http://bit.ly/2Xa6qAu

El primer layout que vamos a ver es el **BorderLayout**. Este gestiona los elementos en cinco posiciones diferentes: norte, sur, derecha, izquierda y centro.

En cada posición podremos poner un único elemento, pero si queremos poner más de uno, podremos colocar un panel y agregar ahí todos los elementos que queramos, pudiendo cambiar el layout de ese panel también, haciendo un anidamiento de layouts.

5.1. Flowlayout y GridLayout

Otro de los layouts que más vamos a utilizar es el **FlowLayout**. Con este, todos los **elementos** que agreguemos estarán siempre **centrados según el tamaño de la ventana** que tengamos. En caso de que no quepan todos los elementos, se pondrán unos debajo de los otros para ir adaptándose.

> **¡! Sabías que...**
>
> Un ejemplo muy sencillo para entender este layout es pensar en una calculadora. La distribución de sus botones tiene la misma disposición que uno de estos layouts.

El **GridLayout** nos va a permitir **maquetar los elementos como si fueran una tabla**. Cuando agregamos un GridLayout a un panel, podremos elegir el número de filas y columnas que queramos que tenga. Para ello seleccionamos el layout y, en la ventana de propiedades, podremos elegirlo con las propiedades *rows* y *columns*.

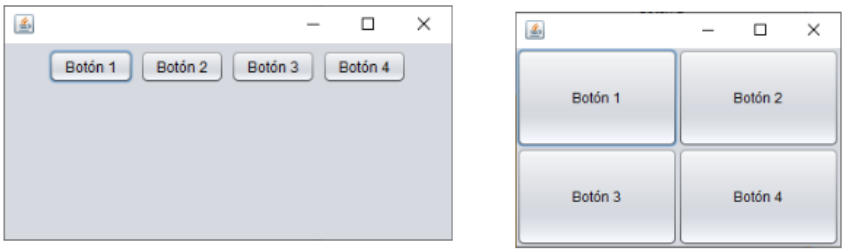


Fig.6. FlowLayout y GridLayout.

/ 6. Ventanas de información

Hasta el momento, todos los mensajes de errores, notificaciones y demás los hemos estado mostrando por consola, cosa que, para quien está desarrollando el programa, no es inconveniente, pero para el usuario de la aplicación, no es efectiva, ya que en ese momento la consola no se verá.

Este problema se puede solucionar con las **ventanas de diálogo predefinidas**. Se trata de ventanas que pueden aparecer en cualquier punto de nuestro programa y muestran un mensaje de error, una notificación o cualquier otra cosa que sea una tarea muy sencilla de realizar.

La primera ventana de diálogo predefinida que vamos a ver es la **ventana de información**. Mediante esta podremos notificar un mensaje al usuario.

Para crear esta ventana escribiremos el siguiente código:

```
String mensaje = "Mensaje de información";
JOptionPane.showMessageDialog(this, mensaje);
```

Como se puede observar, se emplea la clase JOptionPane, la cual tendremos que importar. El funcionamiento es muy sencillo, utilizamos el método showMessage, al que le pasamos dos parámetros:

- **'this'**: Es la ventana padre de la que dependerá la ventana de información. Será siempre este valor.
- **'mensaje'**: Es el mensaje que queremos mostrar.

El resultado de ejecutar ese código es el mostrado en la figura 7:

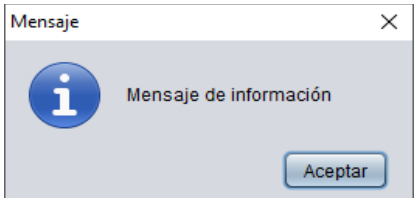


Fig.7. Diálogo de información.

Este tipo de ventanas se conocen como **modales**, esto quiere decir que hasta que no cerremos el mensaje, no podremos interactuar con nuestro programa. Otra manera de crear ventanas modales puede ser utilizando la clase JDialog. La siguiente figura muestra un ejemplo básico de cómo crear una ventana modal en Java:

```
public static void main(String[] args) {
        JFrame frame = new JFrame("Ventana Principal");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(300, 200);
        frame.setLocationRelativeTo(null);

        JButton button = new JButton("Abrir Ventana Modal");
        button.addActionListener(e -> {
            // Crear una ventana modal
            JDialog dialog = new JDialog(frame, "Ventana Modal", true); // El tercer parámetro indica si es modal
            dialog.setSize(200, 100);
            dialog.setLocationRelativeTo(frame);

            JLabel label = new JLabel("Esta es una ventana modal");
            dialog.add(label);

            dialog.setVisible(true); // Mostrar la ventana modal
        });

        frame.add(button);
        frame.setVisible(true);
    }
```

Fig.8. Creación de ventana modal.

/ 7. Caso práctico 1: "Problemas de adaptación a la ventana"

Planteamiento: Los layouts son de gran importancia para organizar y controlar la disposición de los componentes en una interfaz gráfica. Permiten establecer la apariencia y el comportamiento de los elementos, influyendo en la experiencia del usuario. Su empleo facilita la creación de interfaces intuitivas y visualmente atractivas, mejorando la usabilidad y la interacción con la aplicación.

Nudo: Una vez creada la aplicación y las ventanas, se ha ejecutado el programa y, al maximizar la primera ventana, los elementos se han quedado exactamente en el mismo sitio, dejando una ventana prácticamente vacía. ¿Qué ha pasado?, si se han creado las ventanas correctamente, ¿por qué crees que ha ocurrido esto?

Desenlace: Lo más normal, cuando se está aprendiendo a crear interfaces gráficas, es que se descuide uno de los pasos más importantes.

Más allá de colocar todos los elementos necesarios en su lugar correspondiente, es frecuente olvidar que las ventanas pueden (y seguro que lo harán) cambiar de tamaño. Cuando una ventana cambia de tamaño, bien sea para ser más pequeña o más grande, los elementos de la misma se han de adaptar, si no, quedará una ventana con los elementos cortados o con gran cantidad de espacio vacío.

Para solucionar esto, disponemos de los layouts. Si usamos el layout apropiado, cuando cambiemos el tamaño de nuestras ventanas observaremos que los elementos de las mismas se adaptarán automáticamente a ellas, sin tener que hacer nada al respecto.



Fig.9. Diseño de interfaces con layouts.

/ 8. Ventanas de peligro y error

Algo muy útil a la hora de utilizar las ventanas de diálogo predefinidas es la posibilidad de **mostrar mensajes de peligro**, es decir, **advertencias para el usuario**. Mediante la ventana de peligro podremos hacer esto de una forma muy sencilla.

Para crear esta ventana escribiremos el siguiente código:

```
String mensaje = "Mensaje de peligro";
String titulo = "Peligro";
JOptionPane.showMessageDialog(this, mensaje,
titulo, JOptionPane.WARNING_MESSAGE);
```

La siguiente imagen (figura 10) muestra el resultado de ejecutar ese código:

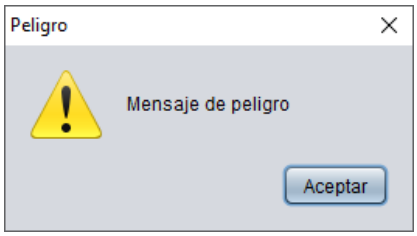


Fig.10. Diálogo de peligro.

Al método showMessage() se le pasan los siguientes parámetros:

- **'this'**: Es la ventana padre de la que dependerá la ventana de información.
- **'mensaje'**: Mensaje que queremos mostrar.
- **'titulo'**: Título que queremos que tenga la ventana que vamos a mostrar.
- **JOptionPane.WARNING_MESSAGE**: Con esto indicamos que es una ventana del tipo peligro y mostrará el icono que le corresponde.

La siguiente ventana predefinida que vamos a ver es la ventana de error. Mediante esta, podremos informar de los errores ocurridos en el programa al usuario. Este tipo de ventana la podremos utilizar, por ejemplo, dentro de los bloques *catch* para informar de las excepciones.

Para crear esta ventana, escribiremos el siguiente código:

```
String mensaje = "Mensaje de peligro";
String titulo = "Peligro"; JOptionPane.
showMessageDialog(this, mensaje, titulo,
JOptionPane.WARNING_MESSAGE);
```

La siguiente imagen (figura 11) muestra el resultado de ejecutar ese código:

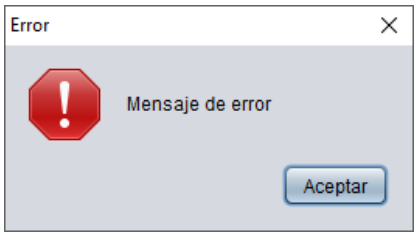


Fig.11. Diálogo de error.

El método showMessage() tendrá los mismos valores exceptuando el tipo, que, en este caso, en lugar de peligro indica un error y por tanto cambia a JOptionPane.ERROR_MESSAGE.

/ 9. Ventana personalizada y preguntas

Ya hemos visto que existen ventanas de diálogo con formas predefinidas, pero podemos crear nuestras propias ventanas de diálogo de forma personalizada. Para ello podremos tener el siguiente código:

```
// Creamos el icono para mostrar
Stringr_icono = "imagenes/emoticono_sonrisa.png";

ImageIconicono = new ImageIcon(getClass().getResource(r_icono));

// Título personalizado, icono personalizado JOptionPane.
showMessageDialog(this, "Mensaje de texto normal.","Título",
JOptionPane.INFORMATION_MESSAGE, icono);
```

La siguiente imagen (figura 12) muestra el resultado de ejecutar ese código:



Fig.12. Diálogo personalizado.

En el código se crea un objeto de la clase ImageIcon a partir de la ruta del icono que queremos mostrar y luego utilizamos el método showMessage, al que le pasamos los cuatro parámetros vistos hasta ahora más el icono que se muestra en la ventana.

La personalización de las ventanas va más allá de simplemente cambiarles el icono. Podemos crear una ventana en la que aparezca una pregunta y el usuario pueda elegir una respuesta, obtenerla y actuar en consecuencia.

Para ello podremos tener el siguiente código:

```
// Texto de las opciones para responder
Object[] opciones = {"Si", "No"};

int respuesta = JOptionPane.showOptionDialog(this,

"¿Desea guardar los cambios realizados?", "Pregunta", JOptionPane.
YES_NO_CANCEL_OPTION,JOptionPane.QUESTION_MESSAGE,

null, opciones, opciones[0]);

System.out.println("Has respondido " + respuesta);
```

La siguiente imagen (figura 13) muestra el resultado de ejecutar ese código:

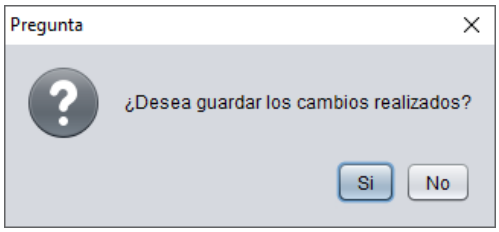


Fig.13. Diálogo personalizado de pregunta.

En el código, primero creamos un array con las posibles respuestas a mostrar y luego utilizamos el método showOptionDialog, al que le pasamos los parámetros visto hasta ahora más:

- **JOptionPane.YES_NO_CANCEL_OPTION**: Con el que indicamos que es una ventana del tipo pregunta.
- **'opciones'**: Serán las opciones a mostrar.
- **'opciones[0]'**: Será la opción que aparecerá señalada.

/ 10. Ventanas para apertura y cierre de ficheros

Seguramente todos hemos utilizado multitud de programas que permiten realizar la opción de abrir un fichero. Para eso, muestran una ventana que nos permitirá navegar por el sistema de ficheros de nuestro ordenador, pudiendo seleccionar el fichero que queramos abrir.

Este tipo de ventanas son otro ejemplo de ventana que ya está hecha y lista para que la llamemos cuando sea necesario.

Para ello necesitamos implementar el código que se muestra en la siguiente imagen (figura 14).

```
// Creamos una ventana JFileChooser,para seleccionar un fichero
JFileChooser fileChooser = new JFileChooser();
// Hacemos que aparezca un fichero por defecto
fileChooser.setSelectedFile(new File("fichero.txt"));
// Cambiamos el título de la ventana
fileChooser.setDialogTitle("Selecciona un fichero");
// Agregamos dos filtros al selecctor de ficheros
FileFilter filter = new FileNameExtensionFilter("Ficheros de texto", "txt");
fileChooser.addChoosableFileFilter(filter);
FileFilter filter2 = new FileNameExtensionFilter("Ficheros java", "java");
fileChooser.addChoosableFileFilter(filter2);
// Mostramos la ventana de seleccionar el fichero
int resultado = fileChooser.showOpenDialog(null);
// Si se ha seleccionado un fichero, si se pulsa cancelar no se ejecuta
if (resultado == JFileChooser.APPROVE_OPTION) {
    // Obtenemos la ruta del fichero seleccionado
    String ruta = fileChooser.getSelectedFile().getAbsolutePath();
}
```

Fig.14. Código de selección de fichero.

Además de poder crear una ventana de selección de fichero, otra que podemos crear es la ventana que aparece en las opciones de guardar, como en cualquier programa. Esta ventana nos permitirá seleccionar un fichero para guardar un contenido que tengamos en nuestro programa.

Para ello escribiremos el código mostrado en la siguiente imagen (figura 15):

```
// Creamos una ventana JFileChooser, para seleccionar un fichero
JFileChooser fileChooser = new JFileChooser();
// Hacemos que aparezca un fichero por defecto
fileChooser.setSelectedFile(new File("fichero.txt"));
// Cambiamos el título de la ventana
fileChooser.setDialogTitle("Selecciona un fichero");
// Mostramos la ventana de seleccionar el fichero
int resultado = fileChooser.showSaveDialog(null);
// Si se ha seleccionado un fichero, si se pulsa cancelar no se ejecuta
if (resultado == JFileChooser.APPROVE_OPTION) {
    // Obtenemos la ruta del fichero
    String ruta = fileChooser.getSelectedFile().getAbsolutePath();
}
```

Fig.15. Código guardar.

Podemos ver que el código es muy parecido al de la ventana de abrir. A este tipo de ventanas también podremos agregarle tantos filtros de ficheros como deseemos.

El resultado de la ejecución de este código es el mostrado a continuación, en la figura 16.

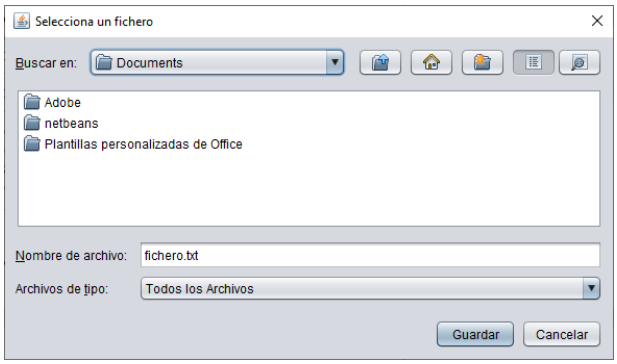


Fig.16. Ventana para guardar un fichero.

/ 11. Caso práctico 2: "Abuso de los menús"

Planteamiento: Los menús de interfaces gráficas se utilizan para proporcionar opciones y funcionalidades adicionales a los usuarios de una aplicación considerándose en diferentes situaciones, como:

- **Cuando se desea organizar las opciones de una aplicación**: Los menús permiten agrupar las diferentes funcionalidades de una aplicación en categorías lógicas, lo que facilita la navegación y el acceso a las diferentes opciones.

- **Cuando se desea proporcionar una interfaz intuitiva y fácil de usar**: Los menús de interfaces gráficas en Java permiten crear una interfaz amigable para los usuarios, ya que les proporcionan una forma estructurada de acceder a las diferentes funcionalidades de la aplicación.

- **Cuando se desea mejorar la experiencia del usuario**: Los menús facilitan la interacción con la aplicación al proporcionar botones, campos de texto y otros componentes visuales que permiten al usuario realizar acciones específicas de manera más sencilla.

Nudo: A primera vista se podría decir que se trata de una herramienta que solo aporta beneficios al sistema que lo implementa, sin embargo, no todas las aplicaciones que usamos la tienen. ¿Qué piensas al respecto? ¿Crees que hay que usar los menús para cualquier funcionalidad que tengamos en nuestra aplicación? ¿O, por el contrario, hay que plantearse su uso?

Desenlace: Es cierto que los menús son algo muy sencillo de utilizar y que nos pueden sacar de más de un apuro cuando no sepamos dónde poner una opción. En los programas, efectivamente, se siguen utilizando, pero poco a poco están desapareciendo en ciertos entornos, como en los programas que utilizan una pantalla táctil, ahí un menú puede ser algo muy poco útil. En estos casos, es mejor usar botones y estudiar detenidamente qué funcionalidades deberemos poner. Algunas situaciones más podrían ser cuando la aplicación es simple y no requiere muchas opciones, cuando la interfaz gráfica es demasiado compleja, cuando se busca una interfaz más moderna y visualmente atractiva o cuando se necesita una interfaz más interactiva y dinámica:

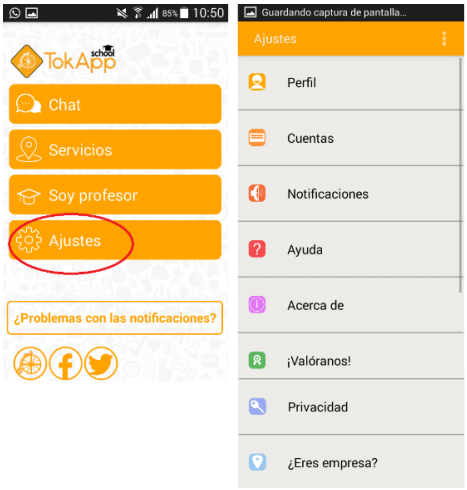


Fig.17. Menús.

/ 12. Menús

Mediante el uso de menús podremos agregar a los programas el típico menú y los submenús que aparecen en todas las aplicaciones y darles funcionalidad como si de un botón se tratase.

Los menús van a permitir que podamos **agrupar funcionalidades y organizarlas** de una forma sencilla y rápida de encontrar para el usuario.

Los menús se encuentran en la paleta de elementos de la biblioteca SWING en la categoría 'Swing Menus'. Para crear menús en Java tendremos las siguientes clases:

- **JMenuBar**: Representa el menú completo de la aplicación.
- **JMenu**: Representa un menú concreto.
- **JMenuItem**: Representa un ítem de un menú, al que le podremos dar funcionalidad.
- **JCheckBoxMenuItem:** Es un ítem de menú que actúa como una casilla de verificación (checkbox).
- **JRadioButtonMenuItem:** Es un ítem de menú que actúa como un botón de radio (radio button).
- **Separator**:Representa un separador entre menús.

Dentro de un menú podremos crear menús dentro de otros menús, es decir, anidar unos menús dentro de otros, además de los elementos básicos como checkbox, radio button y separadores.

Veamos cómo se llevaría a cabo la **agregación de elementos**:

- **Para agregar un menú a nuestro menú de la aplicación**: Seleccionamos el elemento Menu y lo arrastraremos hasta el menú de la aplicación.

- **Para agregar un ítem a un menú**: Seleccionaremos el tipo de ítem que queremos agregar, MenuItem, MenuItemCheckBox o MenuItem Radio Button, y lo arrastraremos dentro del menú donde queremos que aparezca. Una vez agregado, aparecerá dentro del árbol de componentes del programa y podremos cambiarle el texto a mostrar.

- **Para agregar un separador entre los elementos de un menú**: Elegimos Separador en la paleta de menús y lo arrastramos hacia donde queramos que se coloque.

A los menús, podremos darle una funcionalidad que ejecutarán cuando se pulse sobre ellos, para esto haremos doble clic encima del menú al que queramos darle funcionalidad y se le agregará el evento actionPerformed, en el que escribiremos el código que se ejecutará al pulsar sobre el menú.

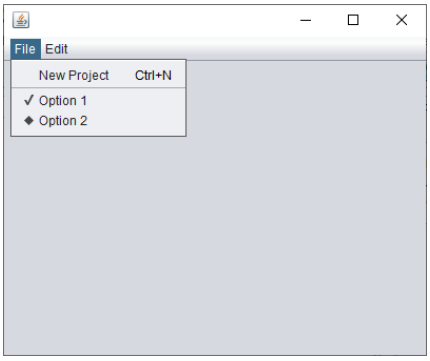


Fig.18. Diferentes menús.

/ 13. Creación y comunicación entre ventanas

Cualquier programa no constará de única ventana, sino que tendrá varias de ellas, lo que hará necesaria una navegación entre ventanas, haciendo que se llamen entre ellas.

Para crear una **ventana nueva** en nuestro programa, deberemos crear un **JFrame nuevo**. Una vez hecho, veremos que tenemos la configuración de elementos a la que estamos acostumbrados y podremos crear una interfaz con todos los elementos ya vistos y sus layouts. Una vez creada la ventana nueva, al ejecutar el programa, veremos que la que se ejecuta es la primera que se creó, ya que es la ventana principal, pero, ¿cómo podremos hacer para que aparezca nuestra nueva ventana?

Para poder lanzar la nueva ventana tendremos que crear un objeto de ella y llamar a los siguientes métodos:

```
// Creamos un objeto de la ventana
NuevaVentana ventana = new NuevaVentana();
// Hacemos que se muestre la ventana ventana.
setSize(200, 200);
//Modificamos el tamaño de la ventana
ventana.setVisible(true);
```

Al trabajar con varias pantallas en un programa será necesario hacer que haya **comunicación** entre ellas. Esto implica hacer **que una pantalla pase información a la siguiente**.

Un ejemplo claro de esto ocurre cuando nos registramos en un programa que consta de varias pantallas de registro y al introducir un dato y pulsar siguiente, dependiendo de ese dato, aparecen unos resultados u otros en la nueva pantalla. Esto es posible ya que la primera pantalla donde introducimos el dato le pasó dicha información a la siguiente pantalla, pudiendo esta mostrar la información correspondiente al dato introducido anteriormente.

Para poder realizar esto necesitaremos seguir los siguientes pasos:

- En la ventana nueva necesitaremos crear una variable de clase con el tipo de dato que le vamos a pasar.
- En la misma ventana crearemos un constructor con parámetros al que le pasaremos el dato que necesitemos. En este constructor deberemos llamar al método initComponentes para que la ventana gráfica se cree correctamente y, por último, escribiremos el código correspondiente a la acción.
- En la ventana principal, la que va a llamar a la nueva, se creará el objeto de la nueva ventana empleando el constructor con parámetros que acabamos de crear. Por último, se llamará al método setVisible de forma normal.

Una vez hecho esto, cuando llamemos a la nueva ventana, le estaremos pasando la información pertinente y se mostrará un resultado dependiendo de ello.



Vídeo 3. "Comunicación entre ventanas"
https://bit.ly/3ohAPsO



/ 14. Resumen y resolución del caso práctico de la unidad

A lo largo de este tema hemos conocido cómo podemos crear un proyecto en NetBeans que tenga interfaz gráfica. Hemos visto también muchos de los elementos, aunque no todos, que podremos utilizar, como los JTextField, los JLabel, los JButton y los JTable. Hemos observado sus características de desarrollo, como hacer que un botón tenga una funcionalidad al pulsarlo; saber, cuando utilizamos una lista de elementos, cuál está seleccionado, o cómo hacer para que los radio button estén agrupados.

Hemos aprendido que todos estos elementos se pueden maquetar mediante los layouts, y que hay diferentes layouts que maquetarán de forma distinta los elementos, según lo necesitemos.

A continuación se ha visto que existen varios tipos de ventanas ya diseñadas y listas para usarse, ventanas de diálogo predefinidas empleadas para mostrar mensajes al usuario, y también hemos conocido la personalización de ventanas que permiten hacer preguntas y obtener las respuestas.

Por último, se ha visto cómo integrar menús con sus respectivos submenús en nuestros programas de una forma muy sencilla y rápida, pudiendo hacer que dispongan de funcionalidad al seleccionarlos. Y hemos abordado también la creación de ventanas, cómo llamarlas y hacerles enviar información.

A continuación podrás ver el esquema resumen del tema.

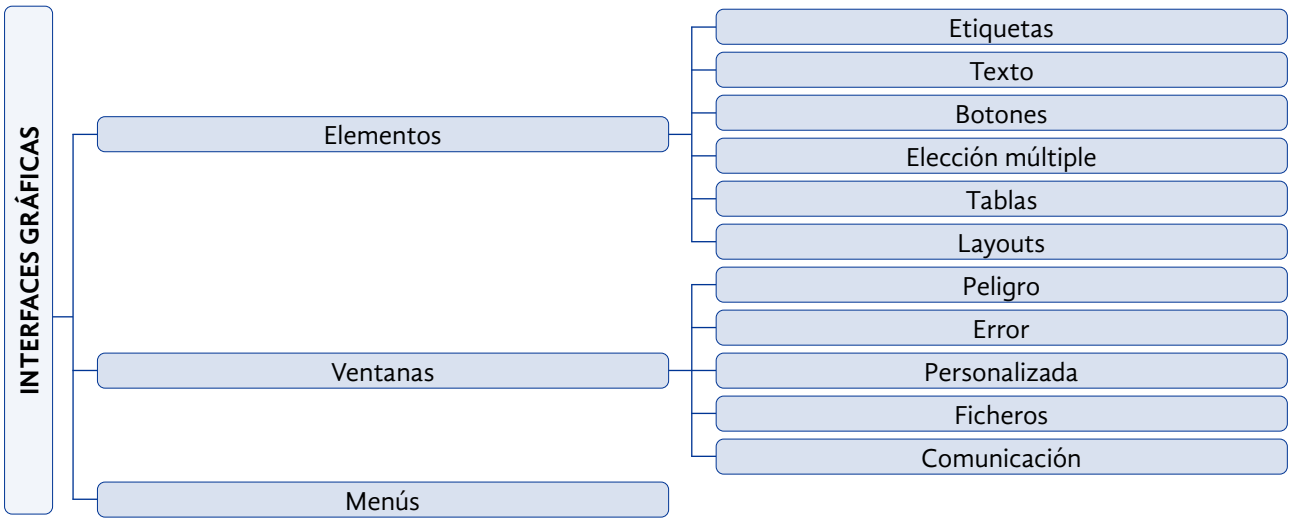


Fig.19. Esquema resumen del tema.

Resolución del caso práctico de la unidad

Cuando empezamos con el diseño de interfaces, lo habitual es tener una única ventana en las aplicaciones, ya que se están dando los primeros pasos y los programas no deberían ser muy complicados.

Al ir avanzando en el estudio, ya notamos que con una única ventana estaríamos muy limitados en el desarrollo, ya que casi cualquier programa con miras a cubrir una necesidad real, por simple y sencillo que sea, va a necesitar de varias ventanas que, incluso, se comuniquen entre ellas.

Para poder crear más de una ventana, tal y como hemos visto en la unidad, basta con crear un nuevo JFrame; al igual que hemos hecho al crear el proyecto, creamos un objeto de la ventana y hacemos que se muestre la ventana. Una vez hecho esto, tendremos disponible una nueva ventana en nuestro proyecto.

/ 15. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos*. Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Bloch, J. (2018). *Effective Java* (3.ª ed.). Addison-Wesley.

Caro, C. M., Ramos, A. N. y Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. Prentice Hall.

Knuth, D. E. (1997). *The Art of Computer Programming* (Vol. 1: *Fundamental Algorithms*) (3.ª ed.). Addison-Wesley.

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S.A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11.ª ed.). McGraw-Hill Education.